

# Assignment: AI Developer

Experience: ~6 months

Expected effort: 10 to 12 hours

Deliverable: Working ML model + API + documentation + code repository

---

## Problem Statement

Build an AI-based route prediction system that predicts the **best route sequence for drivers** on a **daily and weekly basis**.

The system should learn patterns from historical trip data and external signals to recommend:

1. Optimal order of locations to visit
2. Expected travel duration
3. Suggested route for:
  - Daily schedule
  - Weekly schedule
4. Route score (efficiency/confidence)

Use Google Maps APIs and Places APIs.

Output should expose predictions through APIs.

---

## Business Scenario

A field sales company has drivers visiting multiple stores/customers every day.

Drivers often create routes manually, causing:

- Extra fuel cost

- More travel time
- Missed visits
- Unbalanced workload

Your task is to create a lightweight AI system that learns from historical movement and recommends optimized routes.

---

# Technical Requirements

## Mandatory

### Data Collection

Candidate should create/generate sample dataset:

Example:

| Driver | Date       | Stop    | Latitude | Longitude | Visit Time | Visit Duration |
|--------|------------|---------|----------|-----------|------------|----------------|
| D1     | 2026-05-01 | Store A | 23.023   | 72.571    | 09:00      | 25             |
| D1     | 2026-05-01 | Store B | 23.035   | 72.580    | 10:30      | 15             |

Minimum:

- 1000+ trip records
- 10+ drivers
- 50+ locations

Historical data can be:

- synthetic
  - generated
  - publicly available
- 

## Google APIs (Mandatory)

Use:

- Google Maps Platform
- Google Places API

Required usage:

### **Maps**

- Distance Matrix API or Routes API
- Travel duration
- Traffic estimate

### **Places**

- Nearby places
  - Place details
  - Geocoding if required
- 

## **Feature Engineering**

Candidate should create features such as:

### **Time features**

- Day of week
- Hour
- Weekend flag
- Weekly patterns

### **Route features**

- Total distance
- Historical travel time
- Stop count
- Area clustering

### **Driver features**

- Average speed
- Daily visits
- Past route efficiency

### **Location features**

- Store density
  - Region
  - Traffic category
- 

# Model Requirement

Build a small ML model that predicts:

## Daily Prediction

Input:

```
{
  "driver_id": "D1",
  "date": "2026-05-20",
  "locations": [
    "A",
    "B",
    "C",
    "D"
  ]
}
```

Output:

```
{
  "recommended_route": [
    "A",
    "C",
    "B",
    "D"
  ],
  "predicted_time": "3.2 hours",
  "confidence": 0.89
}
```

---

## Weekly Prediction

Input:

```
{  
  "driver_id": "D1",  
  "week": "2026-W20"  
}
```

Output:

```
{  
  "monday": ["A", "B", "C"],  
  "tuesday": ["D", "E"],  
  "wednesday": ["F", "G"],  
  "weekly_distance": "230km"  
}
```

---

## Suggested Modeling Approaches

Candidate can choose any:

Beginner:

- Random Forest
- XGBoost
- K-Means + heuristic route optimization

Intermediate:

- Graph-based optimization
- Reinforcement learning
- Route ranking model

Bonus:

- Small Transformer
- Sequence prediction model
- LSTM route sequence model

Candidate should justify model selection.

---

# API Requirement

Create REST APIs.

Mandatory endpoints:

## **Predict Daily Route**

POST /predict/daily

---

## **Predict Weekly Route**

POST /predict/weekly

---

## **Retrain Model**

POST /retrain

---

## **Health Check**

GET /health

---

Suggested stack:

Backend:

- Python
- FastAPI

ML:

- Scikit-learn
- XGBoost
- TensorFlow/PyTorch

Database:

- SQLite/PostgreSQL
- 

## Folder Structure

project/

```
|— data/  
|— notebooks/  
|— model/  
|— api/  
|— scripts/  
|— tests/  
|— README.md  
|— requirements.txt  
|— dockerfile
```

---

## Evaluation Criteria

| Area                   | Weight |
|------------------------|--------|
| Code quality           | 15%    |
| Google API integration | 20%    |
| Feature engineering    | 15%    |
| Model approach         | 20%    |
| API implementation     | 15%    |
| Documentation          | 5%     |
| Scalability thinking   | 10%    |

---

# Bonus Tasks

Any of these earns extra points:

- ✓ Driver route visualization on map
- ✓ Docker deployment
- ✓ Unit tests
- ✓ Caching Google API calls
- ✓ Route confidence score
- ✓ ETA prediction
- ✓ Dynamic rerouting when traffic changes
- ✓ Model monitoring dashboard